Welcome to ST 308!

# DAY 2: BASICS OF R PROGRAMMING

# Announcement: Reminder

Your course structure looks like the following…

1. Read guided walkthroughs BEFORE class
2. OPTIONAL: Read sections from textbook and short videos that are posted
3. Go through a short lecture in class and do a coding activity together DURING class
4. Attendance is required (two unexcused absences dropped)

Short in-class quiz every couple weeks

Homeworks throughout the semester

# Questions?

# Goals for Today

1. Go over the differences between R, R Studio, and Quarto
2. Introduce R Studio and some of the basics
3. Go over objects, classes, and some of the code functions that will be covered in the activity
4. Install tinytex package in your console
5. Work through the activity together

# What is R and RStudio?

R: Engine




RStudio: Dashboard



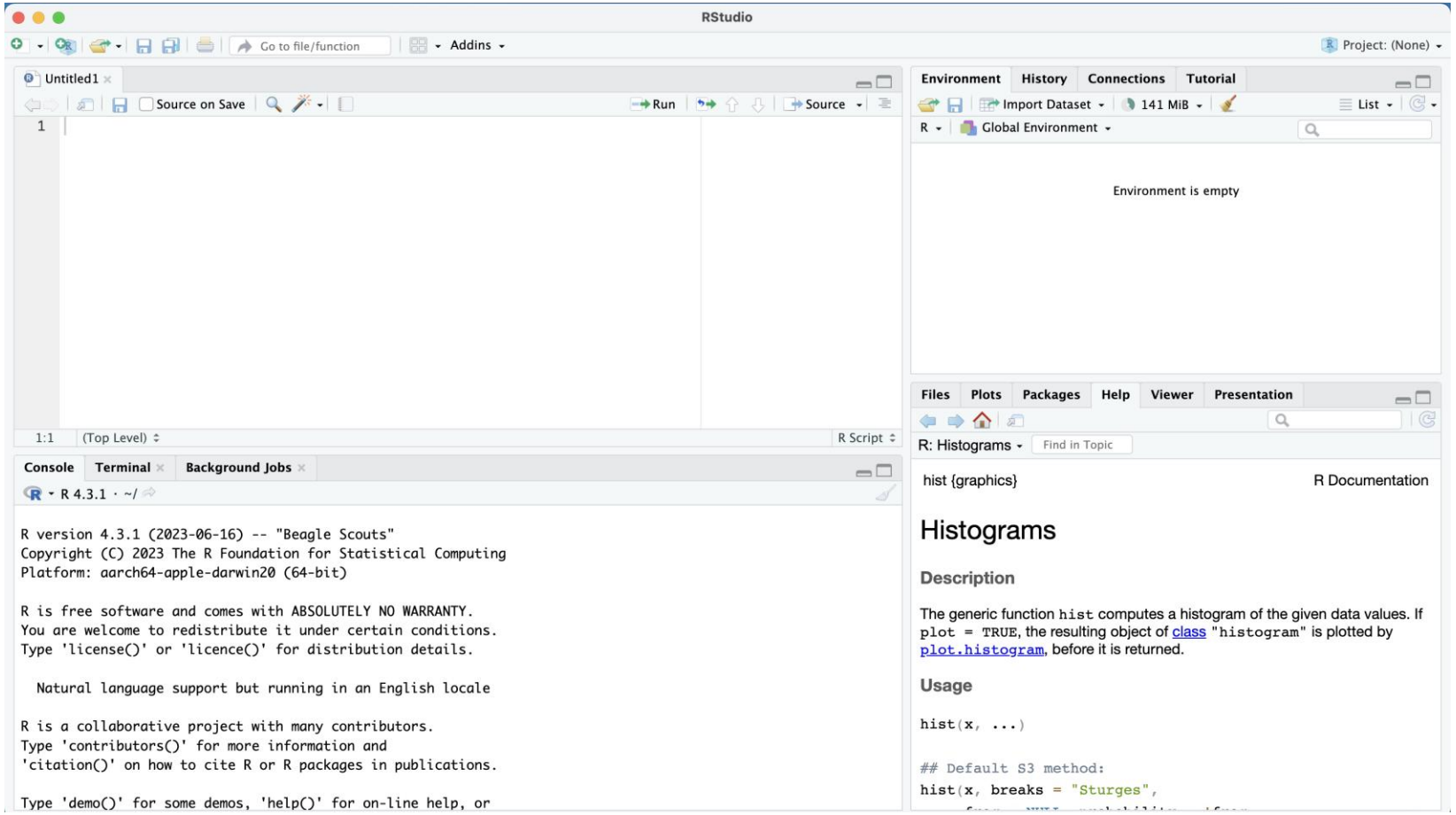
# What is Quarto?



# Four Sections of R Studio

Source

Environment




Console

Files/Help

# Four Sections of R Studio Continued

- **Console:** This is where code will be run, executed, and where you will see error messages pop up.  You can write code here, but it will not be saved
- **Source:** This is where we will write all our code this semester. This allows code to be saved, and we can run the code from the source section to be ran in the console.
- **Environment:** This is where all the objects you have generated along with their values and properties can be seen. You will also be able to import datasets from this section (more on this later this semester)
- **Files/Plots/Help:** This is where you will be able to see your files, any plots that get made, and where you can go to look at the help page for a function.

# Some R Coding Basics

- **= or <-** : You will see both be used to assign something to a variable or object name.
- # : You can use hashtags to add comments to your code (this is extremely useful especially when you are first learning a coding language)
- **function():** put your arguments inside the parenthesis of a function
- Case Sensitivity
  - Spelling and Punctuation Matters!
  - If you assign a variable a name such as “Sports”, you have to spell it with the capital S every time on future lines of code

# In Class Activity 1 Content

# Objects and Classes

- **Objects**: data structures with attributes (**i.e** things that take some sort of values)
- **Classes:** types of *objects* and determine how the objects' values are saved.
  - Vector: 1-dimension object that takes values of all one type, such all numerical values
    - **R Code Example:** x = c(2,4,3,1)
  - List: 1-dimension object that can take values of different types, such as numerical values AND categories
  - Data Frame: 2-dimension object with a certain number rows and columns, very similar to a matrix or array.
    - will be used a lot in this course because this is how most data in the real world is organized.

# Code Functions and Notation Covered

- **str():** provides the structure of an object, meaning it will show the object's value as well as its class
- **c():** stands for concatenate or combine, this combines objects to form a vector
- $: allows you to access the different columns within a *data frame*
- *- :* removes whatever you put after it from the object
- : allows you to access a sequence of entries from an object
- head(): this will only print/output the first 6 entries of an object

# Accessing/Indexing Elements From An Object

- There are ways to access elements inside of an object using square brackets. Let's start with vectors!

Let's say we create a vector in R called x with 4 numbers: x = c(4,6,7,5)

- Vectorname[#]: this is the format of how to access elements from a vector
- x[3]: This is telling R to go into vector x and look for the 3rd observation, which is 7.
- x[1:3]: This is telling R to grab the 1st through 3rd observations = 4, 6, and 7
- x[-2]: The negative is telling R to REMOVE that observation from the vector, so we are left with the 1st, 3rd, and 4th observations = 4, 6, and 5
- **Challenge Question**:
  1. What is x[-c(1,4)] doing and what is the answer?

# Accessing/Indexing Elements From An Object Continued

- We can do the same thing but with a data frame instead of an object.

Let's suppose we have a dataset with 50 rows and 5 columns, and we assign this to an object called "animals".

- datasetname[rows, columns]: since a data frame is 2 dimensions, we need two arguments not one. This is the format of how it would look.
- animals[, 1]: Notice how there is a comma. If you do not put anything before the comma, this tells R that you want it to return ALL of the rows. So, this would return column 1 with all of the rows
- animals[1,]: This is the opposite. Now, the comma is second. This means we are returning the 1st row with ALL of the columns
- animals[5,3] This returns the 5th row for just the 3rd column

# Accessing/Indexing Elements From An Object Continued

Let's keep doing examples with this example animals dataset:

- animals[c(1:10), 4]: This will output the 1st through the 10th observations for the 4th column.
- animals[c(1,25,50), c(2,3)]: This will output the 1st, 25th, and 50th observations for the 2nd and 3rd columns
- animals[, -c(1,5)]: This will REMOVE the 1st and 5th columns leaving the dataset with ALL the rows with the 2nd, 3rd, and 4th columns
- **Challenge Question:**
  1. What is animals[-c(45:50), -c(2,4)] doing and what is the answer?

# Other Ways to Access Columns

There are other ways to access columns in a data frame. Suppose the 2nd column in the dataset is has a variable name: *Type*

There are three different ways we can access the column  Type using code

1. **animals[,2]:** This is the way we covered already. We know it is the 2nd column so we can do this to access ALL the rows with just the 2nd column.
2. **animals[, "Type"]:** This is another way to access it using indexing. This is used when there is many columns and we don't know the exact number column that Type is
3. **animals$Type:** This is a way to directly access the column using the dollar sign. This is more used as arguments inside functions and allows you to bypass indexing.

# Rendering to a PDF in Quarto

We are about to start the first activity of the semester, but first…

To be able to compile PDFs, you will also need to download TinyTex. You can do so by running the following code in your Console.

```
install.packages("tinytex")
tinytex::install_tinytex()
```

We will do this now before starting the activity!

# ICA 1